Git Version Control: Internals, Branching, Collaboration and Practical Demonstration

Madhura Korgaonkar
Deapartment of Computer Science and Engineering
Manipal Institute of Technology
Manipal, India
madhura.mitmpl2026@learner.manipal.edu

***Abstract*—This report presents a practical study of Git as a distributed version-control system, undertaken as the second task of the Formula Manipal Driverless task phase. The work covers repository structure, commits, branching, merging, rebasing, remotes, pushing, pulling, merge-conflict resolution, force pushes, stashing, logs, and reference logs. The concepts were studied using the Learn Git Branching interactive environment together with Git documentation and the tutorial supplied for the task. The practical component connects these concepts to the eight Python programs completed in Formula Manipal Task 1, using a local Git repository to demonstrate committing, branching, merging, synchronizing with a remote repository, and maintaining project history.**

Keywords—Git, version control, commit, branch, merge, rebase, remote, reflog, stash, GitHub

I. INTRODUCTION

Git is a distributed version-control system designed to record changes to files while allowing multiple lines of development to be maintained and combined. Unlike a system that stores only a sequence of complete project snapshots, Git represents history using objects and references. This makes operations such as branching and switching between development lines lightweight. The objective of this task was to understand Git beyond basic add-and-commit usage. The required learning covered branching, pushing, pulling, commits and their storage, logs, reference logs, merging, rebasing, merge-conflict resolution, force pushes, and stashing. Learn Git Branching was used to visualize Git history, while Git documentation and the supplied tutorial were used as supporting references.

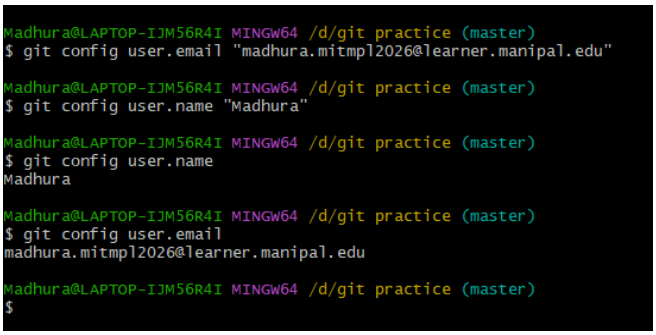
II. GIT FUNDAMENTALS

The main Git object types are blob, tree, commit, and annotated tag. A blob stores file contents, a tree maps names to blobs or subtrees, and a commit points to a tree and its parent history. Objects are identified by content-based object IDs, allowing unchanged content to be reused across different project states.

A. Repositories and Commits

A Git repository stores the history of a project. A Git repository can be understood through three working areas: the working tree, the staging area (index), and the repository database. The working tree contains the files being edited. The staging area records the exact changes selected for the next commit. The repository database stores Git objects and references.

To link an email and username to all your repositories use the ‘git config --global user.email "you@example.com" ' and ‘git config --global user.name "Your Name" ‘, commands respectively. To set identity only in a repository or directory omit --global and run the above commands in that particular directory.



Creating a local repository:-

git init:- This puts the files in that folder in the working directory.

git status:- used to check the status of the files (whether they are in the working directory or staging area).

git add filename1 filename2..:- used to add the untracked files or files in the working directory to the staging area. This adds only the specified files to the staging area.

git add . :- used to add all the files in the working directory to the staging area.

git rm –cached filename… :- used to put a file in the staging area back into the working directory, when no commits have been made.

To keep files certain files in the directory without committing them or adding them to the local repository, create a file named .gitignore and type in the file names to be ignored.

Git does not track empty directories. To track an empty folder, create a .gitkeep file in that folder.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

```
Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice
$ git init
Initialized empty Git repository in D:/git practice/.git/

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        ppt.pptx
        text document.txt
        text.txt
        word document.docx

nothing added to commit but untracked files present (use "git add" to track)
```

```
Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (master)
$ git add text.txt

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   text.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        ppt.pptx
        text document.txt
        word document.docx


Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (master)
$ git add .

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   ppt.pptx
        new file:   text document.txt
        new file:   text.txt
        new file:   word document.docx


Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (master)
$
```

A commit represents a recorded state of the project at a particular point in time. Instead of simply being a copy of the entire project, a commit records information that allows Git to identify the project state and its relationship to earlier commits. Commits form a history that can be inspected and moved through when required. A normal commit has one parent, whereas a merge commit can have two or more parents.

The command git commit is used to put the files from the staging area to the repository database. Git commit is not used with any arguments and hence using this command commits everything present in the staging area.

git commit: - used to commit all the files in the staging area.

git commit -m "message": - used to commit all the files in the staging area whilst adding a commit message.

git restore --staged filename :- used to put a file in the staging area back into the working directory, after first commit has been made.

```
Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (master)
$ git commit -m "Add files"
[master (root-commit) d322b0f] Add files
 4 files changed, 2 insertions(+)
 create mode 100644 ppt.pptx
 create mode 100644 text document.txt
 create mode 100644 text.txt
 create mode 100644 word document.docx

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (master)
$ git status
On branch master
nothing to commit, working tree clean

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (master)
$
```

a. Logs

git log – displays a list of all the commits made so far

git log –oneline – displays a list of all the commits in short and in a line

git log –oneline~4 – displays the last 4 commits

```
Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (branch1)
$ git log
commit 738c9996dbb9577bb120d746c9f45fc6a2c74c19 (HEAD -> branch1)
Author: Madhura <madhura.mitmpl2026@learner.manipal.edu>
Date:   Sat Sep 12 01:28:36 2026 +0530

    Add the remaining files

commit d322b0fc34ca439b8e9b67c257d691458823b041 (master, branch)
Author: Madhura <madhura.mitmpl2026@learner.manipal.edu>
Date:   Fri Sep 11 22:39:57 2026 +0530

    Add files

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (branch1)
$ git log --oneline
738c999 (HEAD -> branch1) Add the remaining files
d322b0f (master, branch) Add files
```

b. Tagging

A tag is a name or label attached to a specific commit. They are commonly used to mark versions or releases. Both tags and branches point to a commit but a branch moves on making a new commit whereas a tag stays fixed. Therefore, tags are used to mark specific locations or give a permanent name to a commit.

git tag – shows the tags

git tag -a v1.0 -m "message" – annotated tag

git tag v1.1 – lightweight tag

```
Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (branch1)
$ git tag -a v1 -m "message"

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (branch1)
$ git tag v0

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (branch1)
$ git tag
v0
v1

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (branch1)
$ git log
commit 738c9996dbb9577bb120d746c9f45fc6a2c74c19 (HEAD -> branch1, tag: v1, tag: v0)
Author: Madhura <madhura.mitmpl2026@learner.manipal.edu>
Date:   Sat Sep 12 01:28:36 2026 +0530

    Add the remaining files

commit d322b0fc34ca439b8e9b67c257d691458823b041 (master, branch)
Author: Madhura <madhura.mitmpl2026@learner.manipal.edu>
Date:   Fri Sep 11 22:39:57 2026 +0530

    Add files

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (branch1)
$ git checkout branch
Switched to branch 'branch'

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (branch)
$ git tag v3

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (branch)
$ git log
commit d322b0fc34ca439b8e9b67c257d691458823b041 (HEAD -> branch, tag: v3, master)
Author: Madhura <madhura.mitmpl2026@learner.manipal.edu>
Date:   Fri Sep 11 22:39:57 2026 +0530

    Add files

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (branch)
$
```

B. *Branching*

A branch is a movable reference to a line of development. Branches make it possible to work on different features or experiments without immediately changing another line of development. Branches are used to work different features without directly modifying the main branch. Once the work is complete and finalized these branches can be merged with the main branch.

git branch branch - creates a new branch

git branch: - shows a list of all the existing branches

git switch branch: - used to switch to the branch whose name is specified

git switch -c branch - creates and switches to that branch

git branch -d branch - deletes the specified branch

git checkout branch– selects the specified branch

*git checkout branch^* - used to move up 1 commit

git checkout branch~4 – used to move up 4 commits

```
Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (master)
$ git branch
* master

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (master)
$ git branch branch

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (master)
$ git branch
  branch
* master

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (master)
$ git switch branch
Switched to branch 'branch'

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (branch)
$ git switch -c branch1
Switched to a new branch 'branch1'

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (branch1)
$ git branch
  branch
* branch1
  master
```

III. BRANCH INTEGRATION

A. *Merging*

Merging combines the histories of branches. When the histories allow it, Git may perform a fast-forward merge by moving a branch reference forward. When the branches have diverged, Git can create a merge commit that records the integration of the two lines of development. Merging is prefferred over rebasing when commit history is to be preserved.

To merge a branch with master branch: -

git switch master – switches to master branch

git merge branchname: - merges the branch specified with the master branch by creating a merge commit.

```
Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (branch)
$ git switch master
Switched to branch 'master'

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (master)
$ git merge branch1
Updating d322b0f..738c999
Fast-forward
 excel.xlsx | Bin 0 -> 7945 bytes
 git.docx   | Bin 0 -> 13319 bytes
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 excel.xlsx
 create mode 100644 git.docx

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (master)
```

B. *Rebasing*

Rebasing changes the base of a sequence of commits by replaying those commits on another base. It can produce a more linear project history, but because rebased commits are recreated, the operation changes commit identities. This makes care necessary when rebasing commits that have

already been shared with others. Rebasing is used to obtain a clean commit hsitory as it leaves out the merge commits.

To merge a feature branch with the master branch without creating merge commits, first switch to the feature branch then using 'git rebase master', rebase the branch onto master. Then switch back to the master branch and use the 'git merge feature' command to merge the two branches. Since the two branches are already aligned, there are no merge commits created.

git rebase -i master – rebases the current selected branch onto master

git rebase -i master branch – rebases the specified branch onto master

```
Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (branch2)
$ git rebase -i master branch2
Successfully rebased and updated refs/heads/branch2.

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (branch2)
$ git log
commit 903bf062728cf54f49f5136643631 81a3b930d84 (HEAD -> branch2)
Author: Madhura <madhura.mitmpl2026@learner.manipal.edu>
Date:   Sat Sep 12 01:47:03 2026 +0530

    Add all files

commit dab7eb378a06e22599b0508284084 74e0bf1f46e
Author: Madhura <madhura.mitmpl2026@learner.manipal.edu>
Date:   Sat Sep 12 01:46:41 2026 +0530

    Add text1 file

commit 738c9996dbb9577bb120d746c9f45fc6a2c74c19 (tag: v1, tag: v0, master, branch1)
Author: Madhura <madhura.mitmpl2026@learner.manipal.edu>
Date:   Sat Sep 12 01:28:36 2026 +0530

    Add the remaining files

commit d322b0fc34ca439b8e9b67c257d691458823b041 (tag: v3, branch)
Author: Madhura <madhura.mitmpl2026@learner.manipal.edu>
Date:   Fri Sep 11 22:39:57 2026 +0530

    Add files

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (branch2)
$ git switch master
Switched to branch 'master'

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (master)
$ git merge branch2
Updating 738c999..903bf06
Fast-forward
 text1.txt | 0
 word.docx | 0
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 text1.txt
 create mode 100644 word.docx

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (master)
$ git log
commit 903bf062728cf54f49f5136643631 81a3b930d84 (HEAD -> master, branch2)
Author: Madhura <madhura.mitmpl2026@learner.manipal.edu>
Date:   Sat Sep 12 01:47:03 2026 +0530

    Add all files

commit dab7eb378a06e22599b0508284084 74e0bf1f46e
Author: Madhura <madhura.mitmpl2026@learner.manipal.edu>
Date:   Sat Sep 12 01:46:41 2026 +0530

    Add text1 file

commit 738c9996dbb9577bb120d746c9f45fc6a2c74c19 (tag: v1, tag: v0, branch1)
Author: Madhura <madhura.mitmpl2026@learner.manipal.edu>
Date:   Sat Sep 12 01:28:36 2026 +0530

    Add the remaining files
```

C. Resolving Merge Conflicts

A merge conflict occurs when Git cannot automatically combine changes, commonly because different branches modify overlapping parts of a file. Resolving a conflict involves examining the conflicting changes, deciding what the final content should be, removing the conflict markers, and then completing the merge. Conflict resolution is an important part of collaborative Git workflows.

IV. REMOTE REPOSITORY OPERATIONS

A. Pushing and Pulling

A remote repository provides a shared location for Git history. Pushing transfers local commits to a remote repository, while pulling obtains changes from a remote and integrates them into the local repository. The Remote section of Learn Git Branching was used to understand the relationship between local branches, remote-tracking references, and remote operations.

git push – pushes all the commits from the local repository to the remote repository.

git clone – creates a local repository of a remote repository.

git fetch – fetches the commits from the remote repository but does not merge them with the local repository.

git pull – fetches and merges the commits from the remote repository to the local repository.

B. Stashing

Stashing temporarily stores local working changes so that the working tree can be cleaned without committing unfinished work. The saved changes can later be reapplied. This is useful when a developer needs to switch branches or address another task before the current changes are ready for a commit

git stash - stashes the files in working directory

git stash list - shows stashed files

git stash pop - brings the stashed files back to working directory

git stash apply - files remain in stash and are also copied to working directory

```
Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (master)
$ git stash
Saved working directory and index state WIP on master: 903bf06 Add all files

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (master)
$ git stash apply
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   git.txt
        new file:   github.docx


Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (master)
$ git stash list
stash@{0}: WIP on master: 903bf06 Add all files

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (master)
$ git stash pop
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   git.txt
        new file:   github.docx

Dropped refs/stash@{0} (57b21051c33d1cf4eef86243b436442b905a52a4)

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (master)
$ git stash list

Madhura@LAPTOP-IJM56R4I MINGW64 /d/git practice (master)
$
```

V. CONCLUSION

The Git task was completed by working through the Main and Remote sections of Learn Git Branching and by performing Git operations on a local repository. The work covered branching, commits, logs, merging, rebasing, conflict resolution, pushing, pulling, force pushes, and stashing. The Task 1 solutions were also pushed to the repository. The practical exercises established a stronger understanding of how Git manages development history and supports collaboration.

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to Formula Manipal for providing me with the opportunity to work on this task and learn more about Git and version control. I would also like to thank the Formula Manipal Driverless team and mentors for their guidance and support throughout the task.

This task helped me understand Git beyond just basic commands, including branching, commits, merging, rebasing, remote repositories, and conflict resolution. I am grateful for the practical experience and the opportunity to apply these concepts while working on the task.

REFERENCES

[1] Learn Git Branching, "Learn Git Branching," available at: https://learngitbranching.js.org/.

[2] GitHub Docs, "About Git," available at: https://docs.github.com/en/get-started/using-git/about-git.

[3] YouTube, "Git Tutorial," available at: https://www.youtube.com/watch?v=HVsySz-h9r4].